

Flutter iOS Development with Xcode

Step-by-Step Setup Guide

This guide explains how to take your existing Flutter app and run it on a real iPhone using Xcode on macOS. It is written for your current workflow: code can stay on Windows, but the iOS build and signing must happen on a Mac.

1. What you need

- A Mac with macOS and the latest compatible Xcode installed.
- Your Flutter project copied to the Mac.
- Your iPhone and a USB cable.
- An Apple ID added in Xcode. A personal Apple account can be enough for testing on your own device.
- Flutter installed on the Mac and visible in Terminal.

Important: you cannot build and sign a real iOS app from Windows alone. Apple requires Xcode on macOS for the final iPhone build and signing step.

2. Move your project to the Mac

Copy the full Flutter project folder to the Mac using Git, a USB drive, cloud storage, or a network share. Open Terminal on the Mac and go into the project directory.

```
cd /path/to/your/flutter/project
flutter doctor
flutter pub get
```

If flutter doctor shows missing iOS or Xcode components, fix those first before opening the iOS project.

3. Open the iOS workspace in Xcode

From the root of the Flutter project, open the generated iOS workspace. Use the workspace, not the standalone Xcode project file.

```
open ios/Runner.xcworkspace
```

In Xcode, the left sidebar should show the Runner project. Select the Runner target to edit app settings.

4. Prepare the iPhone

- Connect the iPhone to the Mac with USB.
- Unlock the iPhone and tap Trust if prompted.
- Enable Developer Mode on the iPhone if iOS asks for it.
- Wait until the device appears as an available run target in Xcode.

5. Add your Apple ID in Xcode

In Xcode, go to Xcode > Settings > Accounts and sign in with your Apple ID. Once added, Xcode can create a development team for signing your app.

6. Configure signing

In Xcode, click the top-level Runner project, then select the Runner target, then open Signing & Capabilities.

- Turn on Automatically manage signing.
- Choose your Team from the Team dropdown.
- Set a unique Bundle Identifier, for example: com.madeplus.smartshoe
- Keep the same Team for all relevant app targets if Xcode shows more than one.

If Xcode says the bundle identifier is already used, change it to a more unique value.

7. Build and run on the iPhone

At the top of Xcode, choose your connected iPhone as the run destination. Then press the Run button.

Xcode will build the Flutter iOS app, sign it, install it on the device, and launch it. The first build can take a while because Xcode resolves dependencies and prepares the app.

8. Flutter CLI commands you can use on the Mac

```
flutter pub get
open ios/Runner.xcworkspace
flutter run
flutter build ios --no-codesign
```

Use flutter run for day-to-day testing after Xcode signing is already configured. Use flutter build ios --no-codesign if you only want to generate the iOS build artifacts without signing.

9. Recommended first-time workflow for your app

- 1 Copy the updated Flutter project from Windows to the Mac.
- 2 Run flutter pub get.
- 3 Open ios/Runner.xcworkspace in Xcode.
- 4 Set Team and Bundle Identifier in Signing & Capabilities.
- 5 Connect the iPhone 13 Pro Max.
- 6 Press Run in Xcode.
- 7 Test Bluetooth permissions, connection flow, and live sensor data on the real device.

10. Common problems and fixes

Device does not appear in Xcode

- Unlock the iPhone.
- Tap Trust on the device if prompted.
- Enable Developer Mode on the iPhone.
- Use a working USB cable and port.

Signing error or No provisioning profile found

- Make sure Automatically manage signing is enabled.
- Choose the correct Team.
- Use a unique Bundle Identifier.
- Wait a minute and let Xcode finish signing setup.

App installs but will not launch

- Check whether iPhone Developer Mode is enabled.
- Rebuild from Xcode after reconnecting the phone.
- Look in the Xcode debug console for the first runtime error.

Flutter packages fail on iOS build

- Run flutter pub get again.
- Re-open Runner.xcworkspace instead of Runner.xcodeproj.
- If needed, clean and rebuild from the project root.

```
flutter clean
flutter pub get
open ios/Runner.xcworkspace
```

11. After the first successful install

Once the app is installed on the phone, future development is easier. You can keep editing Flutter code, then rebuild from Xcode or with flutter run on the Mac. For deeper iOS work such as capabilities, bundle settings, permissions, and device debugging, use Xcode.

12. Notes for Bluetooth testing

- Always test Bluetooth behavior on a real iPhone, not just a simulator.
- Confirm iOS permission prompts appear correctly on first launch.
- Check that reconnect, disconnect, and background-close behavior are stable on the phone.
- If BLE reconnect still fails after app restarts, compare phone behavior against a BLE scanner app to separate app-side and sensor-side issues.

13. Source references

- [Flutter documentation - Build and release an iOS app](#)
- [Flutter documentation - iOS setup for Flutter apps](#)
- [Apple Developer Documentation - Xcode](#)
- [Apple Developer Documentation - Running your app in Simulator or on a device](#)
- [Apple Developer Documentation - Enabling Developer Mode on a device](#)

Prepared for your Flutter smart-shoe project so you can move from Windows development to iPhone testing through Xcode on a Mac.